

Assignment 4 - Bitcoin Miner

M11315016 - 林品安

1 Implementation

1.1 Key CUDA features in SHA256 Implementation

- Constant Memory for Hash Constants

```
1 __device__ __constant__ static const WORD k[64] = {  
2     0x428a2f98, 0x71374491, 0xb5c0fbcf, ...  
3 };
```

- The `__device__ __constant__` qualifiers store the SHA256 constant array in CUDA constant memory because:
 - Constants are frequently accessed by all threads
 - Constant memory is cached, reducing memory bandwidth
 - Allows single-cycle broadcasts when all threads read same value
 - Limited to 64KB but provides faster access than global memory

- SHA256 Transform Function

```
1 __device__ __host__ void sha256_transform(SHA256 *ctx,  
2                                           const BYTE *msg)
```

- The transform function uses `__device__ __host__` qualifiers to:
 - Enable execution on both GPU and CPU
 - Process single 512-bit block of data
 - Apply compression function core of SHA256
 - Allow result verification between CPU and GPU computations

- Main SHA256 function

```
1 __device__ __host__ void sha256(SHA256 *ctx,  
2                                const BYTE *msg,  
3                                size_t len)
```

- The main hash function with `__device__ __host__` qualifiers:
 - Handles complete message processing
 - Manages message padding and length addition
 - Initializes hash values
 - Coordinates multiple block transformations
 - Ensures consistent results between GPU and CPU execution

1.2 Key CUDA features in main file (hw4.cu)

- Aligned memory structures using `__align__(8)` to ensure proper memory access:

```
1 typedef struct __align__(8) _block {
2     unsigned int version;
3     unsigned char prevhash[32];
4     unsigned char merkle_root[32];
5     unsigned int ntime;
6     unsigned int nbits;
7     unsigned int nonce;
8 } HashBlock;
```

- CUDA kernel for parallel nonce searching:
- Each thread calculates a hash for a unique nonce value
- `tid` is calculated as `blockIdx.x * blockDim.x + threadIdx.x`, giving a unique ID to each thread
- Memory Management
 - Host data is transferred to the GPU using `cudaMemcpy`.
 - The `find_nonce` kernel utilizes global device memory for block data and target values, ensuring all threads can access required inputs.
 - The result is stored in a `MiningResult` structure, with atomic operations ensuring thread safety.

- Kernel code

```
1 __global__ void find_nonce(const HashBlock block,
2                             const unsigned char* target_hex,
3                             unsigned int start_nonce,
4                             MiningResult* result)
5 {
6     int tid = blockIdx.x * blockDim.x + threadIdx.x;
7     if (result->found) return;
8
9     HashBlock my_block = block;
10    my_block.nonce = start_nonce + tid;
11    // ... hash calculation and comparison
12 }
```

- Host code

```
1 while (!h_result.found && start_nonce <= 0xFFFFFFFF - THREADS_PER_BLOCK *
2     NUM_BLOCKS) {
3     find_nonce<<<NUM_BLOCKS, THREADS_PER_BLOCK>>>(block, d_target, start_nonce,
4         d_result);
5     cudaDeviceSynchronize();
6     cudaMemcpy(&h_result, d_result, sizeof(MiningResult), cudaMemcpyDeviceToHost);
7
8     start_nonce += THREADS_PER_BLOCK * NUM_BLOCKS;
9 }
```

- Performance measurement using CUDA events:

```
1 cudaEvent_t start, stop;
2 CUDA_CHECK(cudaEventCreate(&start));
3 CUDA_CHECK(cudaEventRecord(start));
4 // ... mining operations
5 CUDA_CHECK(cudaEventRecord(stop));
6 CUDA_CHECK(cudaEventSynchronize(stop));
7 float milliseconds = 0;
8 CUDA_CHECK(cudaEventElapsedTime(&milliseconds, start, stop));
```

2 Parallelization and Optimization Techniques

- **Grid-Stride Loop Pattern:** Using configurable thread and block counts:

```
1 #define THREADS_PER_BLOCK 256
2 #define NUM_BLOCKS 2048
3 // Used in kernel launch:
4 find_nonce<<<NUM_BLOCKS, THREADS_PER_BLOCK>>>(...)
```

- **Memory Optimizations:**

- Local block copying to reduce global memory access
- `tid` calculated in the kernel ensures that each thread computes a hash for a unique nonce, effectively dividing the workload across all available threads

```
1 HashBlock my_block = block; // Local copy
2 my_block.nonce = start_nonce + tid;
```

- block and target data are transferred to the GPU memory once at the start, avoiding repeated data transfers during mining

```
1 cudaMalloc(&d_block, sizeof(HashBlock));
2 cudaMalloc(&d_target, 32);
3 cudaMalloc(&d_result, sizeof(MiningResult));
4
5 cudaMemcpy(d_block, &block, sizeof(HashBlock), cudaMemcpyHostToDevice);
6 cudaMemcpy(d_target, target_hex, 32, cudaMemcpyHostToDevice);
7 cudaMemset(d_result, 0, sizeof(MiningResult));
```

- **Compiler Optimizations:** Using advanced flags in Makefile:

- `-arch=sm_86` specifies the target GPU architecture to ensure compatibility with the hardware for optimal performance
- `-dc` generates device code to allow linking of separate compilation units

```
1 NVFLAGS = -std=c++11 -O3 -dc -arch=sm_86
```

3 Experiments

- As `THREAD_PER_BLOCK` increases, the throughput consistently improves while the execution time decreases across all `NUM_BLOCKS` values. This suggests better resource utilization with higher thread counts.
- Increasing `THREAD_PER_BLOCK` from 64 to 256 consistently improves throughput. At `NUM_BLOCKS=1024`, throughput increases from 280.13 MH/s (`THREAD_PER_BLOCK=64`) to 340.83 MH/s (`THREAD_PER_BLOCK=256`), a $\approx 21.7\%$ improvement.

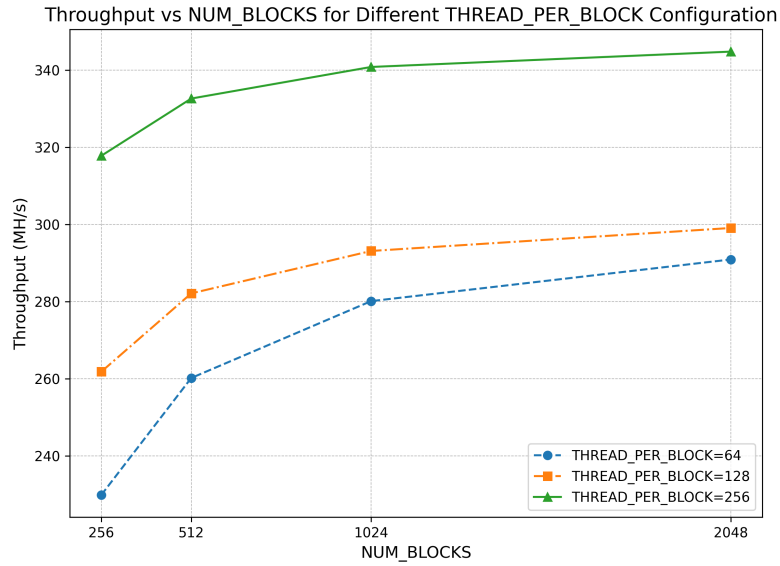


Table 1: Throughput vs NUM_BLOCKS for Different THREAD_PER_BLOCK Configurations

4 Advanced CUDA Features

- **Atomic Operations:** Thread-safe nonce updates:

```

1 if(cuda_little_endian_bit_comparison(...) < 0) {
2     if(atomicCAS(&result->found, 0, 1) == 0) {
3         result->nonce = my_block.nonce;
4     }
5 }

```

- **Error Checking:** Comprehensive error handling macro:

```

1 #define CUDA_CHECK(call) { \
2     cudaError_t err = call; \
3     if (err != cudaSuccess) { \
4         fprintf(stderr, "CUDA error in %s:%d: %s\n", \
5             __FILE__, __LINE__, \
6             cudaGetErrorString(err)); \
7         exit(EXIT_FAILURE); \
8     } \
9 }

```

- **Performance Monitoring:** Throughput calculation:

```

1 printf("Mining throughput: %.2f MH/s\n",
2 (start_nonce / 1000000.0f) / (milliseconds / 1000.0f));

```